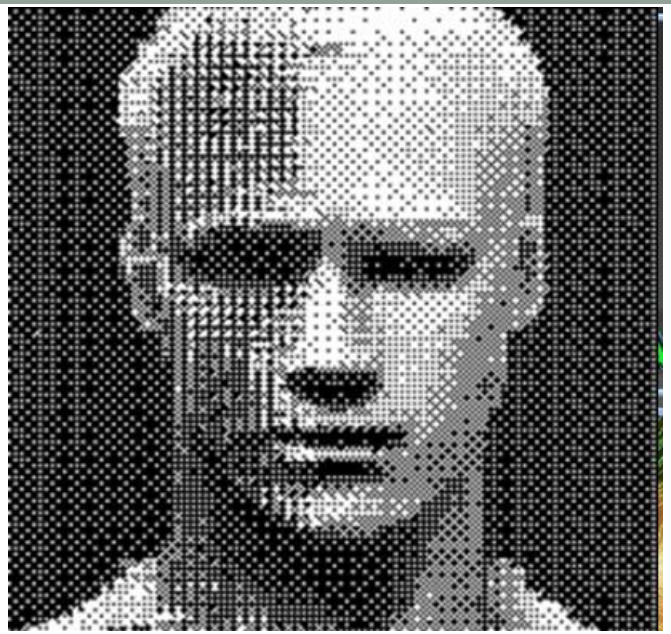


# DIGITAL IMAGE PROCESSING

Dr. Manar Donia

CMPn332 - CMPN332 & CMP 432

# Relationships between pixels

- Pixel relationship helps define the structure, content, and the image's visual quality.
- They are vital in many tasks such as image enhancement, segmentation, edge detection, and feature extraction.
- key types of relationships between pixels:
  - Neighboring
  - Connectivity (adjacent)
  - **Labelling ( connected regions)**
  - **Distance measures**

# Labelling ( connected regions)

## Labelling Algorithm based on 4-adjacency

Input: Labelling a particular pixel $(\mathrm{row}, column)$

- let us assume the start is from the top left pixel then

Start: Visit Pixels from Left to Right then top to bottom

If: $f(r,c) \notin \xi$ then replaced it with 0 and move to the next one $f(\mathrm{r}, c+1)$.

else : check on the labels of previous neighbor

**if:** previous neighbor pixels has the same label then assign the current pixel to same label

**else if:** some of neighbors in different labels then put the pixel in any label of them and record that all these labels are equivalent

**else:** start a new label and assign that pixel to the new label

End if

Continue to visit until the last pixel visited

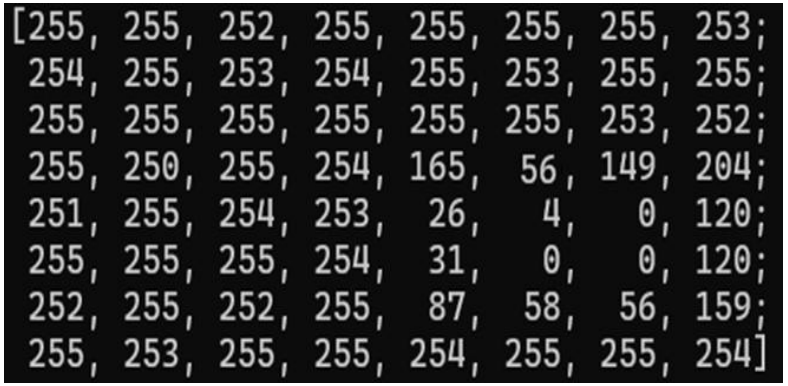
Join all marked equivalent labels to single label

End

# Labelling ( adjacent regions)

Example :

Consider the image shown, find connected regions given the connectivity set $\xi = \{26,4,0,31,87,58,56\}$




- **Answer**

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 2 | 1 | 1 | 0 |
| 0 | 0 | 0 | 0 | 2 | 1 | 1 | 0 |
| 0 | 0 | 0 | 0 | 2 | 1 | 1 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

- L1=L2
- L1={ f(3,5), f(4,4), f(4,5), f(4,6), f(5,4), f(5,5), f(5,6), f(6,4), f(6,5), f(6,6)}

# Distance measures

- Distance measures between pixels are crucial in image processing for different tasks that involve object recognition, and image segmentation along with others.
- These distance measures help quantify how "far" or "close" two pixels are.
- The most common distance metrics
  - Euclidean distance
  - City Block distance(D4)
  - Chessboard (D8)
  - Dm distance
- Each has its own strengths depending on the problem you are trying to solve in image processing :

# Distance measures

Euclidean distance: is the straight-line distance between two pixels.

- often used when you need to measure the actual physical distance between two pixels.
- The Euclidean distance between p(x, y) and q(s, t) is :

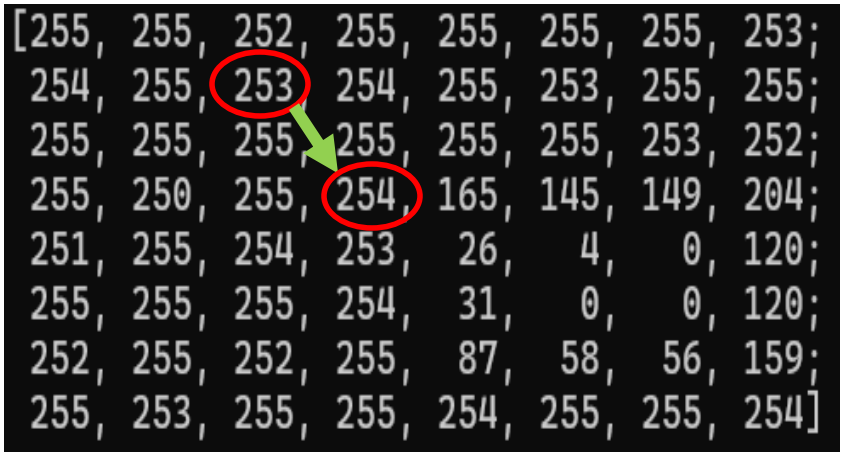
$$D_e(\mathrm{p},\mathrm{q}) = \sqrt{(x-s)^2 + (y-t)^2}$$

Example :
Compute the Euclidean distance between two pixels p and q with coordinates p(1,2) and q(3,3)

**Answer**

$$D_e(p,q) = \sqrt{(1-3)^2 + (2-3)^2}$$

$$D_e = \sqrt{5}$$



# Distance measures

City Block distance($D_4$): measures the distance between two pixels by only allowing movement horizontal and vertical directions.

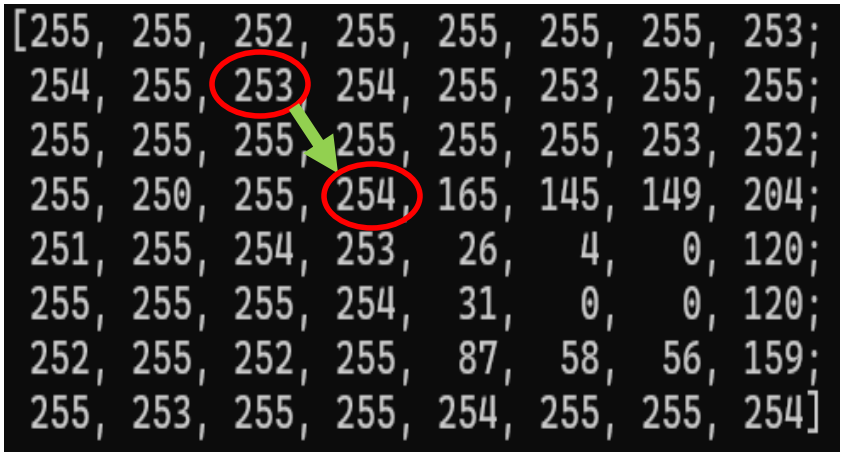
- often used for maze-solving algorithms
- The City Block distance ($D_4$) between p(x, y) and q(s, t) is:

$$D_4(p,q) = |x-s| + |y-t|$$

Example :
Compute the City Block distance between two pixels p and q with coordinates p(1,2) and q(3,3)

**Answer**

$$D_4(p,q) = |1-3| + |2-3|$$
$$D_4 = 3$$



# Distance measures

Chessboard ($D_8$): measures the distance between two pixels by considering both horizontal/vertical and diagonal movements.

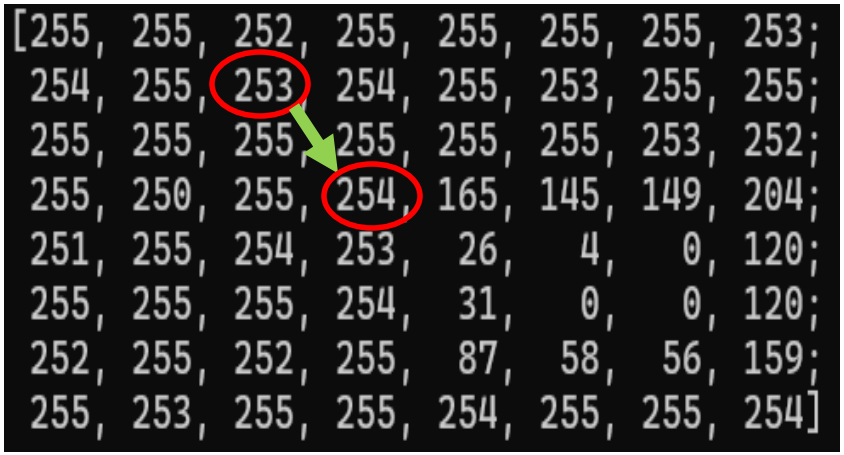
- often used in **pathfinding, pattern recognition** and **object matching**
- Chessboard ($D_8$) between p(x, y) and q(s, t) is:

$$D_8(p,q) = \max(|x-s|, |y-t|)$$

Example :
Compute the Chessboard distance between two pixels p and q with coordinates p(1,2) and q(3,3)

**Answer**

$$D_8(p,q) = \max(|1-3|, |2-3|)$$
$$D_8 = 2$$



# Distance measures

$D_m$ distance is defined as the shortest m-path between the pixels.

- This distance depend on the values of the pixels along the path, as well as the values of their neighbors.

**Example :**

For the shown image matrix find the distance $D_m$ between two pixels p and q with coordinates p(2,0) and q(0,2) giving the connectivity set $V = \{1\}$

| | | |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |

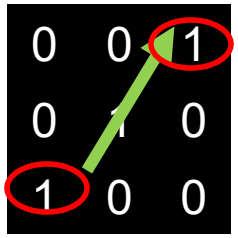
**Answer**

**m-adjacency neighbors** of p(2,0) $N4 \in V = \emptyset$

$N_D(2,0) \in V = \{f(1,1)\}$ ***check on connectivity***

p(2,0) $N4 \in V = \emptyset$

$N_4(2,0) \cap N_4(1,1) = \emptyset$ $so\ it's\ m\ adjacent$




**m-adjacency neighbors** of $f(1,1)$ $N4 \in V = \emptyset$

$N_D(1,1) \in V = \{q(0,2)\}$ ***check on connectivity***

$f(1,1)$ $N4 \in V = \emptyset$

$N_4(1,1) \cap N_4(0,2) = \emptyset$ $so\ it's\ m\ adjacent$

The length of the shortest m-path= {p(2,0), p(1,1) , p(0,2)}

$D_m$=n-1=2

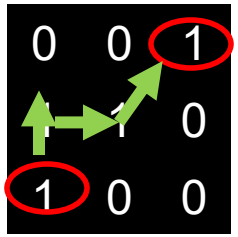
# Distance measures

**Example2 :**

For the shown image matrix find the distance $D_m$ between two pixels p and q with coordinates p(2,0) and q(0,2) giving the connectivity set $V = \{1\}$

| | | |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 1 | 0 |
| 1 | 0 | 0 |

**Answer**




**m-adjacency neighbors** of p(2,0) $N4 \in V = \{f(1,0)\}$

$N_D(2,0) \in V = \{f(1,1)\}$ ***check on connectivity***

$p(2,0)\ N4 \in V = \{f(1,0)\}$

$f(1,1)\ N4 \in V = \{f(1,0)\}$

$N_4(2,0) \cap N_4(1,1) = \{f(1,0)\}$ `so it's not m adjacent`

**m-adjacency neighbors** of $= f(1,0)\ N4 \in V = \{f(1,1)\}$

$N_D(1,0) \in V = \emptyset$

**m-adjacency neighbors** of $= f(1,1)\ N4 \in V = \emptyset$

$N_D(1,1) \in V = q(0,2)$

$f(1,1)\ N4 \in V = \emptyset$

$N_4(1,1) \cap q(0,2) = \emptyset$ *so it's m adjacent*

The length of the shortest m-path= {p(2,0), $f(1,0)$, $f(1,1)$ , $q(0,2)$}

$D_m$=n-1=4-1=3





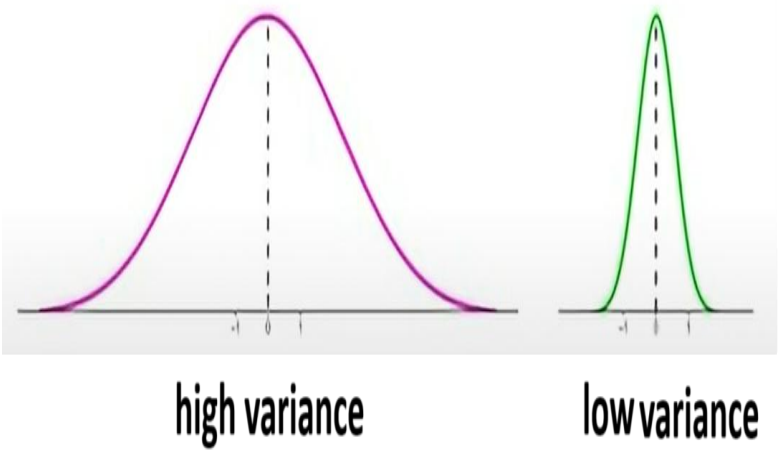


# Importance of Statistical Measures in DIP

- **Statistical measures** provides a quantitative methods for analyzing and interpreting the pixel values within an image.
- By using statistical tools, you can extract meaningful information, analyze pixel intensity distributions, spatial relationships, textures, and noise, which are all vital in improving the performance of image processing algorithms.

# Importance of Statistical Measures in DIP

- Assuming Gray level values: $\mathrm{L} = 0 \rightarrow 255$
- If maximum gray level $50$→ Dark image
- If minimum gray level 200→ Bright Image
- If ratio between (Max & Min) = (100,150)→ low dynamic range
- If average $\begin{cases} 50 \rightarrow dark\ image \\ 200 \rightarrow bright\ image \end{cases}$
- Variance $\begin{cases} \text{high} \rightarrow \text{good } contraat \\ Low \rightarrow \text{low } contraat \end{cases}$



# What is the objective of image processing?

- **Image Analysis and Interpretation→** Extract meaningful information from an image for further analysis or decision-making by machines or humans.

- **Image Enhancement→** geometric transformation, Increasing contrast, reducing noise, sharpening details, or adjusting brightness.

- **Image Restoration→** Restore image to its original or an improved state.

- **Image Compression→** Reduce the size of an image file for efficient storage or transmission without losing significant quality

- **Image Segmentation→** Divide an image into meaningful regions or segments to isolate areas of interest for further analysis.

# Image Enhancement

Image enhancement is done by combining geometric transformation and enhancement techniques to improve the visual quality of images using **Cascading Operations**.

Geometric transformation change the spatial arrangement of pixels to correct geometric distortions that occur with non-ideal camera angles.

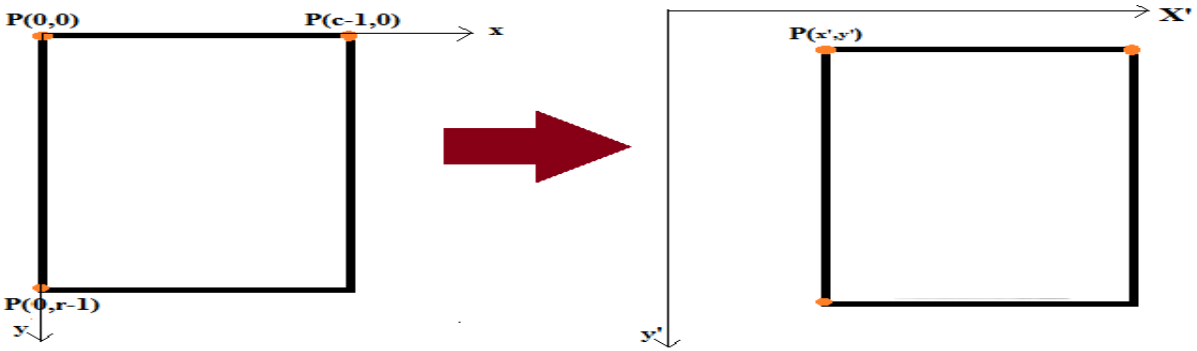
While enhancement techniques modify pixel values without changing their spatial relationships.

**Example**:

- **Geometric Transform:** Scale and rotate the image to a standard size and orientation.
- **Enhance**: Apply enhancement algorithm for noise reduction and contrast adjustment.

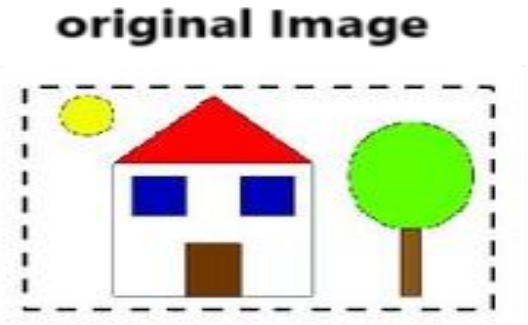
# Geometric Transformation

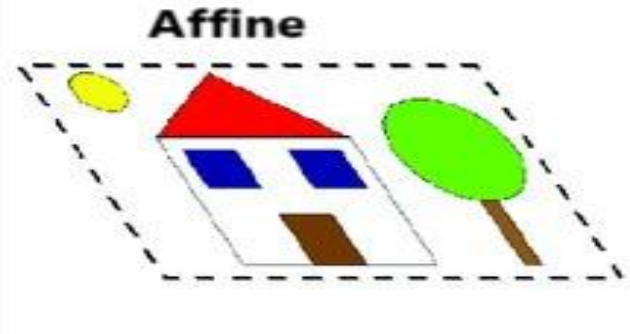
Geometric transformations modify the image's geometry, allowing position, orientation, size, and shape **without altering the underlying pixel values**. **f***(x',y')=**f**(x,y).*




Geometric transformations can be categorized into they differ primarily in how th**Affine transformations** and **non-affine transformations**,ey manipulate the geometry of images.

# Geometric Transformation







| Aspect | Affine Transformations | Non-Affine Transformations |
|---|---|---|
| **Definition** | Preserve points, straight lines, and planes. | Do not preserve lines; may warp or bend the image. |
| **Linear vs. Non-Linear** | Linear transformations. | Non-linear transformations. |
| **Parallelism** | Maintains parallelism (parallel lines remain parallel). | Does not maintain parallelism. |
| **Distance Ratios** | Preserves ratios of distances between points. | Does not preserve distance ratios. |
| **Transformation** | Can be expressed with a matrix in a simple form. | Often requires more complex functions for representation. |
| **Examples** | Rotating an image by 30 degrees, scaling an image to half its size. | Creating a 3D perspective effect, bending an image. |

# Affine Transformations

Affine transformations Maintain geometric properties like parallelism and ratios. They are linear and can be represented by transformation matrices

- **Mathematical Representation**:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b & t_x \\ c & d & t_y \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- Where the matrix elements control the transformation process.

# Affine Transformation techniques

Affine transformations encompass a variety of techniques that preserve the linearity of relationships in an image such as:

- Translation→ Shifts the image content along the x or y axes by specified amounts.

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

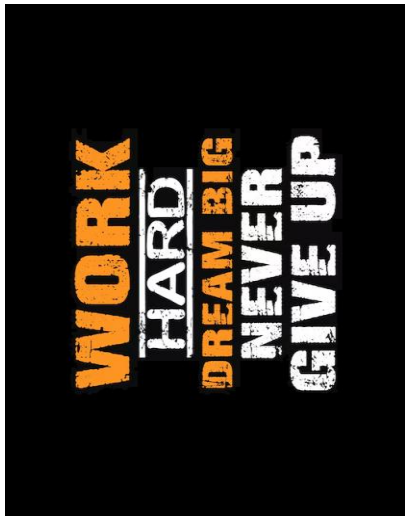


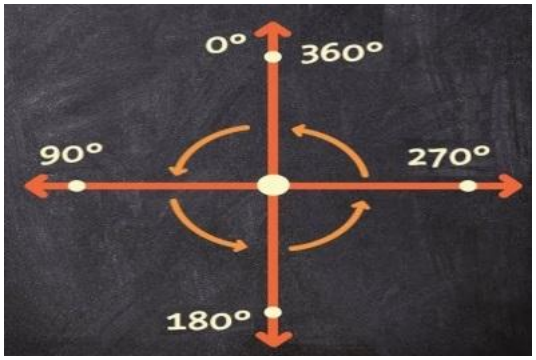



# Affine Transformation techniques

- Rotation→ Rotates the image around a specified point by a given angle.

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$






# Affine Transformation techniques

- Scaling→ Changes the size of the image by a scaling factor, which can be uniform (same factor for both dimensions) or non-uniform (different factors for each dimension).

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$






# Affine Transformation techniques

- Flipping→ Flips the image over a specified axis (horizontal or vertical or diagonal).
  - For Vertical Flip

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$







  - For Horizontal Flip

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$



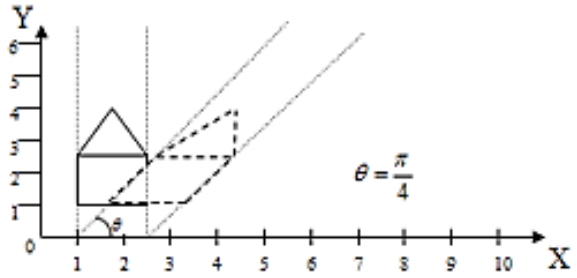



# Affine Transformation techniques

Shrear or Skewing : Deform shape like shifted slices#

shear in x: $\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & sh_x \\ 0 & 1 \end{bmatrix} . \begin{bmatrix} x \\ y \end{bmatrix}$

$x' = x + sh_x . y\,, \quad y' = y$




$Skew_\theta = \begin{bmatrix} 1 & \frac{1}{\tan\theta} \\ 0 & 1 \end{bmatrix}$

shear in y: $\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ sh_y & 1 \end{bmatrix} . \begin{bmatrix} x \\ y \end{bmatrix}$

$x' = x\,, \qquad y' = sh_y . x + y$


Thank you!